

Relazione di progetto “Smash Burger”

Corso di Tecnologie Web A.A. 2025-26

Autori

DA COMPILARE: Nome Cognome

, matricola [MATRICOLA], [email]@studenti.unipd.it

DA COMPILARE: Nome Cognome

, matricola [MATRICOLA], [email]@studenti.unipd.it

DA COMPILARE: Nome Cognome

, matricola [MATRICOLA], [email]@studenti.unipd.it

DA COMPILARE: Nome Cognome

, matricola [MATRICOLA], [email]@studenti.unipd.it (referente)

Sito web

[https://tecweb.studenti.math.unipd.it/\[DA COMPILARE\]](https://tecweb.studenti.math.unipd.it/[DA COMPILARE])

Repository GitHub

<https://github.com/TecWeb2526SB/SmashBurger>

Credenziali di prova

Amministratore: admin / admin

Manager: manager / manager

Cliente: user / user

Indice

1	Introduzione	3
2	Analisi dei requisiti	4
2.1	Analisi utente	4
2.2	SEO	4
3	Progettazione	5
3.1	Architettura dell'applicazione	5
3.2	Schema organizzativo	5
3.3	Tipi di utente	5
3.4	Schema database	5
3.5	Funzionalità principali	6
4	Realizzazione	6
4.1	Struttura e contenuto	7
4.1.1	HTML	7
4.1.2	Popolamento database	7
4.2	Presentazione	7
4.2.1	CSS	7
4.2.2	CSS per la stampa	8
4.2.3	Immagini e icone	8
4.2.4	Font	8
4.2.5	Colori	8
4.3	Comportamento	9
4.3.1	PHP	9
4.3.2	JavaScript	9
4.3.3	Validazione dell'input	10
4.3.4	Sicurezza	10
4.3.5	Errori di navigazione	10
4.4	Accessibilità	10
4.4.1	Aiuti per lo screen reader	11
4.4.2	Compatibilità responsive	11
5	Test effettuati	11
5.1	Accessibilità e validazione	11
5.1.1	Strumenti automatici	12
5.1.2	Test manuali	12
5.2	Integrazione continua (CI/CD)	12
5.3	Analisi dei falsi positivi	13
5.4	Screen reader	13
6	Ambiente di sviluppo	13
7	Organizzazione del gruppo	13
7.1	Divisione dei compiti	13
8	Note	14

1 Introduzione

La presente relazione descrive le metodologie e le scelte progettuali adottate per la realizzazione del progetto del corso di Tecnologie Web (Laurea in Informatica – L31) dell’anno accademico 2025-2026.

Il sito realizzato è dedicato alla catena di hamburgerie “Smash Burger”, un’attività commerciale simulata a scopo didattico con quattro sedi nel Triveneto (Padova, Treviso, Vicenza, Udine). La piattaforma consente la consultazione del menu, l’ordinazione online con modalità di ritiro o consegna a domicilio, la prenotazione della sala eventi e la gestione completa dell’attività tramite un pannello di controllo multilivello.

Il sito offre agli utenti non autenticati:

- consultazione del menu completo con prezzi, descrizioni, allergeni e immagini;
- visualizzazione delle sedi con orari di apertura, indirizzi e contatti;
- informazioni sulla tecnica culinaria dello smash burger e sulla storia aziendale;
- invio di messaggi tramite il modulo contatti;
- cambio del tema grafico (chiaro/scuro).

Gli utenti registrati (clienti) possono inoltre:

- selezionare una sede e comporre un carrello con i prodotti disponibili;
- completare l’ordine scegliendo tra ritiro in sede e consegna a domicilio;
- consultare lo storico degli ordini con ricevute dettagliate;
- prenotare la sala eventi indicando data, orario e numero di persone;
- gestire il proprio profilo, modificare la password ed eliminare l’account.

I manager di sede possono:

- gestire gli orari di apertura settimanali della propria sede;
- abilitare o disabilitare la disponibilità dei singoli prodotti e aggiornare le scorte;
- approvare o rifiutare le prenotazioni della sala eventi;
- visualizzare e gestire gli ordini della propria sede;
- consultare il grafico degli incassi a 30 giorni.

L’amministratore, oltre a tutti i privilegi del manager su ogni sede, può:

- gestire il catalogo prodotti globale (creazione, modifica, eliminazione, caricamento immagini);
- gestire le categorie del menu;
- aggiungere, modificare e disattivare sedi;
- gestire i messaggi ricevuti tramite il modulo contatti;
- gestire gli utenti registrati (promozione, assegnazione sede, attivazione/disattivazione, eliminazione).

2 Analisi dei requisiti

Prima di avviare lo sviluppo, abbiamo analizzato diversi siti web di catene di ristorazione fast-casual e hamburgerie artigianali per identificare le funzionalità e le informazioni principali da includere. A partire da questi riferimenti, abbiamo definito i requisiti funzionali e non funzionali del progetto, documentandoli nel file `docs/ANALISI_REQUISITI.md` del repository.

2.1 Analisi utente

Il sito si rivolge a un pubblico eterogeneo di potenziali clienti di un'hamburgeria, con un'età stimata tra i 16 e i 50 anni. Ci si aspetta che la maggior parte degli accessi avvenga da dispositivo mobile, spesso in mobilità e con l'intento di consultare rapidamente il menu o effettuare un ordine. Il linguaggio adottato è informale ma chiaro, con attenzione alla segnalazione degli allergeni e alla disponibilità di opzioni alternative (vegane, senza glutine).

Sono stati individuati quattro profili utente:

- **Ospite**: utente non autenticato che naviga il sito per consultare menu, prezzi, sedi e orari senza obbligo di registrazione;
- **Cliente (user)**: utente registrato che effettua ordini online (ritiro o domicilio), prenota la sala eventi, gestisce profilo e storico ordini;
- **Manager (manager)**: responsabile di una singola sede (relazione 1-a-1), gestisce orari, disponibilità prodotti, prenotazioni e ordini della propria sede;
- **Amministratore (admin)**: gestore dell'intera catena con accesso completo a catalogo, categorie, sedi, utenti e messaggi contatti.

2.2 SEO

Di seguito le ricerche principali alle quali il sito intende rispondere:

- ricerche dirette sul nome dell'attività ("smash burger padova");
- ricerche relative a opzioni di ristorazione nelle città servite ("hamburger padova centro", "hamburger treviso", "hamburger vicenza", "hamburger udine");
- ricerche legate a modalità di ordinazione ("hamburger da asporto padova", "hamburger a domicilio padova", "ordinare hamburger online");
- ricerche orientate a esigenze alimentari specifiche ("hamburger vegano padova", "allergeni hamburger");
- ricerche informative sulla tecnica culinaria ("che cos'è lo smash burger");
- ricerche relative a eventi privati ("hamburgeria per feste", "eventi privati hamburgeria").

Operazioni svolte per migliorare il posizionamento:

- ogni pagina ha un `<title>` e un `<meta name="description">` univoci e coerenti con il contenuto, definiti centralmente nel file `pagine.php`;
- separazione rigorosa tra struttura (HTML), presentazione (CSS) e comportamento (JavaScript);
- ottimizzazione delle prestazioni tramite compressione delle immagini in formato WebP (sotto 300 KB), caricamento locale dei font, assenza di dipendenze esterne;
- generazione automatica di una `sitemap.xml` per i motori di ricerca.

3 Progettazione

3.1 Architettura dell'applicazione

L'applicazione è stata progettata seguendo il pattern architetturale **Page Controller** con **Transaction Script** e **Template View**, come prescritto dal corso:

- ogni URL corrisponde a un file controller PHP nella radice di `src/`, responsabile dell'instradamento della richiesta;
- la logica di dominio è organizzata in 20 file di funzioni pure (`src/includes/funzioni/`), ciascuno dedicato a un dominio applicativo specifico (carrello, ordini, prenotazioni, catalogo, sicurezza, ecc.), per un totale di 131 funzioni;
- le viste (`src/views/`) ricevono i dati già elaborati tramite `extract($dati)` e si limitano a produrre il markup HTML, senza mai accedere direttamente al database.

Tutte le pagine condividono un template comune composto da `header.php` e `footer.php`, assemblato dalla funzione `mostra_pagina()` che garantisce uniformità strutturale su tutto il sito.

3.2 Schema organizzativo

Il catalogo dei prodotti è organizzato in categorie non sovrapponibili (Hamburger, Contorni, Bevande, Dessert), gestite dinamicamente dall'amministratore. La navigazione del menu è strutturata per categoria, con pagine di dettaglio per ogni singolo prodotto che ne mostrano immagine, descrizione, allergeni e prezzo.

Il sito è composto da 31 pagine, organizzate in quattro aree:

- **Area pubblica** (14 pagine): home, menu, dettaglio prodotto, servizi, chi siamo, sedi, dettaglio sede, contatti, accesso, registrazione, privacy, accessibilità, mappa del sito, cambio tema;
- **Area utente** (7 pagine): area personale, profilo, carrello, pagamento, ricevuta, prenotazione sala eventi, uscita;
- **Area controllo – manager** (5 pagine): ordini, dettaglio ordine, prodotti, dettaglio sede, prenotazioni;
- **Area controllo – amministratore** (5 pagine): scheda prodotto, categorie, sedi, messaggi contatti, utenti.

3.3 Tipi di utente

Come anticipato nella sezione 2.1, sono stati individuati quattro ruoli distinti con privilegi crescenti. Il sistema di autenticazione è basato su sessioni PHP con cookie `HttpOnly` e `SameSite=Lax`. L'accesso alle pagine riservate è controllato dalla funzione `richiedi_permesso()`, che verifica il ruolo dell'utente corrente e restituisce errore 401 (non autenticato) o 403 (non autorizzato) in caso di accesso non consentito.

3.4 Schema database

Il database è stato progettato su MariaDB 10.6 con 12 tabelle in forma normale e 15 vincoli di chiave esterna. Tutti gli importi monetari sono memorizzati in centesimi di euro (tipo `INT UNSIGNED`) per evitare errori di arrotondamento tipici dei tipi a virgola mobile.

Le tabelle principali sono:

- **utenti**: account con ruolo (**cliente**, **manager**, **amministratore**), dati anagrafici e di consegna;
- **sedi**: le quattro sedi della catena con manager associato (relazione 1-a-1);
- **orari_sedi**: orari di apertura settimanali per ogni sede (vincolo UNIQUE su sede+giorno);
- **categorie**: categorie del menu con ordinamento personalizzabile;
- **prodotti**: catalogo prodotti con allergeni, immagine e prezzo in centesimi;
- **disponibilita_prodotti**: matrice sede×prodotto con quantità a magazzino e vincolo CHECK (quantita >= 0);
- **carrelli** e **righe_carrello**: carrello temporaneo associato a un utente e a una sede;
- **ordini** e **righe_ordine**: ordini finalizzati con stato, modalità (ritiro/domicilio), pagamento e storicizzazione del nome e prezzo dei prodotti;
- **prenotazioni**: prenotazioni della sala eventi con data, orario, durata e stato;
- **messaggi_contatto**: messaggi dal modulo contatti con categoria e stato di gestione.

3.5 Funzionalità principali

- registrazione e autenticazione utente con rigenerazione dell'ID di sessione al login;
- consultazione del menu con dettaglio prodotto (ingredienti, allergeni, prezzo, disponibilità per sede);
- procedura d'ordine in 5 passi: scelta sede → composizione carrello → revisione → scelta modalità (ritiro con orario / domicilio con indirizzo) e pagamento → ricevuta;
- gestione concorrente delle scorte: lo scarico avviene in transazione con `UPDATE ... WHERE quantita >= :q`, garantendo atomicità anche in caso di ordini simultanei;
- prenotazione sala eventi con controllo anti-sovrapposizione transazionale sugli intervalli orari;
- pannello di controllo con grafico SVG degli incassi a 30 giorni, generato lato server e accompagnato da una tabella equivalente per l'accessibilità;
- cambio del tema chiaro/scuro tramite endpoint `POST /tema` che persiste la preferenza in un cookie, garantendo il funzionamento anche con JavaScript disabilitato.

4 Realizzazione

La catena “Smash Burger” è un’entità commerciale simulata a scopo didattico. I dati dei prodotti, le descrizioni e i prezzi sono verosimili ma non corrispondono a un’attività reale. Le immagini dei prodotti, delle sedi e degli elementi editoriali sono state **generate tramite strumenti di intelligenza artificiale** al fine di ottenere una resa visiva coerente con il layout del sito. Tali immagini non ritraggono prodotti, luoghi o persone reali. Tutti i file sono stati ritagliati nelle proporzioni corrette per il layout, convertiti in formato WebP e ottimizzati sotto i 300 KB, con testi alternativi `alt` che descrivono accuratamente il contenuto visivo per garantire l’accessibilità.

4.1 Struttura e contenuto

4.1.1 HTML

Il sito è stato sviluppato in HTML5 con sintassi conforme alle regole XML, come richiesto dal corso: tutti gli elementi sono esplicitamente chiusi, gli attributi booleani sono estesi (`required="required"`), e la radice `<html>` dichiara i namespace XML e gli attributi di lingua (`lang="it"` e `xml:lang="it"`).

Il markup è generato dinamicamente da PHP attraverso il sistema di viste Template View. Le 44 viste sono organizzate in 8 sottocartelle tematiche: `account/`, `controllo/`, `errori/`, `informazioni/`, `ordine/`, `prenotazione/`, `pubbliche/` e `template/`. La registrazione centralizzata di tutte le pagine nel file `pagine.php` garantisce che `<title>`, `<meta description>`, breadcrumb, menu di navigazione e controllo degli accessi siano sempre coerenti e aggiornati da un'unica sorgente di verità.

4.1.2 Popolamento database

Il popolamento iniziale del database è effettuato dallo script `schema.sql`, che contiene sia le istruzioni DDL (`CREATE TABLE`) sia i dati di esempio: 4 sedi, 5 categorie, 19 prodotti con descrizioni e allergeni realistici, 3 utenti di prova (admin, manager, user) con password hashate tramite `password_hash()`. L'import è automatizzabile tramite il workflow di deploy (`deploy-tecweb.yml`).

4.2 Presentazione

4.2.1 CSS

La presentazione è gestita da tre fogli di stile distinti, collegati nell'HTML con l'attributo `media` appropriato per ottimizzare il caricamento:

- `stile.css` (2654 righe): foglio principale con reset, variabili CSS (`:root`), layout, componenti semantici, tema chiaro e scuro;
- `mobile.css` (456 righe): adattamenti per schermi con `max-width: 48em` (768px), collegato con `media="screen and (max-width: 48em)"`;
- `stampa.css` (492 righe): foglio di stampa con layout linearizzato, monocromatico e privo di elementi interattivi, collegato con `media="print"`.

Aspetti rilevanti della strategia CSS:

- le variabili CSS (`custom properties`) centralizzano colori, spaziature e ombre, garantendo coerenza e manutenibilità;
- il tema scuro è gestito sia tramite la media query `prefers-color-scheme: dark` (rispetto automatico della preferenza di sistema) sia tramite la classe `.tema-scuro` impostata dal cookie;
- la media query `prefers-reduced-motion: reduce` disabilita tutte le animazioni e transizioni per gli utenti con sensibilità al movimento;
- i layout utilizzano sia Flexbox (23 istanze) sia CSS Grid (25 istanze), quest'ultimo impiegato con attenzione alla complessità di rendering;
- non sono presenti stili inline (`style="..."`) in nessun punto del progetto.

4.2.2 CSS per la stampa

Il foglio di stampa è stato progettato per massimizzare la leggibilità su carta e il risparmio di inchiostro:

- i colori sono convertiti in scala di grigi monocromatica (`--sfondo: #ffffff, --testo: #000000`);
- tutti gli elementi di navigazione, i pulsanti interattivi e le immagini decorative sono nascosti;
- la struttura della pagina è linearizzata per adattarsi al formato cartaceo verticale;
- le ombre grafiche sono rimosse (`--ombra: none`);
- i margini di pagina sono impostati a 2 cm tramite la regola `@page`.

4.2.3 Immagini e icone

La gestione delle risorse grafiche è diversificata per tipologia:

- **Immagini statiche** (`src/images/`): logo, hero della home, immagini editoriali, badge W3C, illustrazioni delle pagine di errore (401, 403, 404, 500) e foto delle facciate delle sedi;
- **Immagini dinamiche** (`src/uploads/prodotti/`): foto dei prodotti caricate dall'amministratore, con validazione MIME (`finfo_file()`), limite dimensionale (300 KB, 300–2000 px) e conversione automatica in WebP;
- **Icone**: SVG sprite unico (`icone.svg`) referenziato tramite la funzione PHP `icona()`;
- **Favicon**: file PNG (`favicon.png`).

La cartella `uploads/` è protetta con un `.htaccess` dedicato che disabilita l'esecuzione di file PHP, impedendo attacchi di upload malevolo.

4.2.4 Font

Il sito utilizza il carattere tipografico **Archivo**, un sans-serif grottesco moderno a forte impatto visivo, distribuito sotto licenza Open Font License. Il font è caricato localmente in tre pesi (regular 400, bold 700, black 900) tramite `@font-face` con direttiva `font-display: swap` per evitare il flash of invisible text.

Stack di fallback: `'Archivo', 'Helvetica Neue', Arial, sans-serif`.

4.2.5 Colori

La palette cromatica è ispirata agli ingredienti dello smash burger, con nomi semantici delle variabili CSS:

Ruolo colore	Tema chiaro	Tema scuro
Sfondo principale	#faf7f2	#171614
Superficie / schede	#ffdf9	#22201d
Testo principale	#1a1a1a	#f2ece4
Senape (accento caldo)	#9c6f00	#e8a100
Ketchup (azione/errore)	#b3181f	#f0555a
Cetriolini (successo)	#3f6b24	#86bd4d
Bordi	#d8cfc2	#413d37
Link visitati	#61208f	#d8adff

Tabella 1: Palette colori dei due temi

I contrasti sono stati verificati per garantire un rapporto minimo di 4.5:1 per il testo normale e 3:1 per il testo grande, in conformità alle WCAG 2.1 livello AA.

4.3 Comportamento

4.3.1 PHP

L'applicazione è sviluppata in PHP 8.1 procedurale puro (0 classi, 131 funzioni distribuite in 20 file). I file principali dell'infrastruttura sono:

- **configurazione.php**: costanti, configurazione sessione, funzioni di URL (`url()`, `risorsa()`, `risalita()`) che producono percorsi puramente relativi per garantire il funzionamento in qualsiasi sottocartella;
- **database.php**: connessione PDO a MariaDB con prepared statements e parametri nominati;
- **risorse.php**: bootstrap che include in ordine tutti i file necessari;
- **pagine.php**: registro unico di tutte le 31 pagine con titoli, descrizioni, ruoli ammessi e posizionamento nei menu.

Nelle sezioni contenenti form, è implementato il ripopolamento automatico dei campi in caso di errore, con messaggi diagnostici specifici che includono il rimedio suggerito (es. "Il campo nome utente deve contenere solo lettere, numeri e trattini").

4.3.2 JavaScript

Il file `script.js` (336 righe) è racchiuso in una IIFE `'use strict'` senza dipendenze esterne. È suddiviso in tre macro-funzioni inizializzate all'evento `DOMContentLoaded`:

1. **Torna su**: gestione del pulsante di ritorno in cima con rilevamento altezza header via `ResizeObserver`, collisione con footer e riposizionamento focus per tastiera;
2. **Moduli AJAX** (Progressive Enhancement): i form con attributo `data-modulo` inviano la richiesta via `fetch` senza ricaricare la pagina; in caso di errore di rete, il fallback è la sottomissione tradizionale tramite `form.requestSubmit()`;
3. **Pagamento**: gestione dinamica del form di checkout che mostra/nasconde i campi di indirizzo in base alla modalità scelta (ritiro/domicilio) tramite `data-attributes` e CSS, senza manipolazione di stili inline.

Progressive Enhancement: tutte le funzionalità essenziali del sito sono operative anche con JavaScript completamente disabilitato. Il cambio tema, ad esempio, è un form POST nativo gestito interamente dal server.

4.3.3 Validazione dell'input

I controlli sull'input sono eseguiti sia lato client (attributi HTML5: `required`, `pattern`, `maxlength`) sia lato server (funzioni PHP dedicate: `contatto_errori()`, `utente_errori_registrazione()`, `prodotto_errori()`, `prenotazione_errori()`, ecc.). Ogni controllo JavaScript ha il corrispettivo identico in PHP, garantendo che la validazione non possa essere aggirata disabilitando lo scripting.

4.3.4 Sicurezza

Sono state implementate le seguenti misure di sicurezza:

- **SQL Injection:** tutte le query utilizzano prepared statements PDO con parametri nominati; nessuna stringa utente è mai concatenata in una query SQL;
- **XSS:** ogni dato inserito nel markup è sanitizzato dalla funzione `e()` che invoca `htmlspecialchars()` con flag `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML5`;
- **CSRF:** ogni form POST include un token generato con `bin2hex(random_bytes(32))`, verificato con `hash_equals()` per resistere a timing attack;
- **Session Fixation:** l'ID di sessione viene rigenerato con `session_regenerate_id(true)` al momento del login;
- **Cookie:** configurati con `httponly = true`, `samesite = Lax` e `secure` dinamico (attivo su HTTPS);
- **Password:** hashate con `password_hash()` (bcrypt) e verificate con `password_verify()`;
- **Upload:** validazione MIME reale con `finfo_file()`, limiti dimensionali, cartella uploads con esecuzione PHP disabilitata via `.htaccess`;
- **Content-Security-Policy:** configurata senza `unsafe-inline` né `unsafe-eval`, con direttive restrittive per `default-src`, `form-action`, `frame-ancestors` e `object-src`;
- **Header HTTP di sicurezza:** `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `X-Frame-Options: SAMEORIGIN`, `Permissions-Policy` restrittiva;
- **Host Header Injection:** validazione regex del dominio in `indirizzo_base()`;
- **Directory Listing:** disabilitato; accesso diretto a `includes/`, `views/` e `database/` bloccato via `.htaccess`.

4.3.5 Errori di navigazione

Sono state create pagine di errore personalizzate per i codici HTTP 401, 403, 404 e 500, ciascuna con un'illustrazione tematica contestualizzata e la navigazione completa del sito, così da non bloccare mai l'utente in un vicolo cieco. Le pagine di errore utilizzano percorsi relativi nelle direttive `ErrorDocument` del file `.htaccess`, come richiesto dal corso.

4.4 Accessibilità

L'obiettivo di conformità è il livello AA delle WCAG 2.1, come stabilito dalla normativa italiana ed europea. Di seguito le scelte implementate:

- **Skip link:** il collegamento "Vai al contenuto" è il primo elemento dopo l'apertura di `<body>`, con gestione JavaScript del focus su `#contenuto`;

- **Torna su:** pulsante con testo nascosto accessibile (`.solo-lettori`) e gestione del focus da tastiera;
- **Navigazione:** le etichette `aria-label` distinguono le aree di navigazione (“Navigazione principale”, “Il tuo account”, “Percorso di navigazione”). La pagina corrente è marcata con `<span aria-current="page">` al posto del link, evitando link circolari;
- **Breadcrumb:** implementato come lista ordinata `<ol>` dentro un `<nav aria-label="Percorso di navigazione">`, senza collegamento per la pagina corrente;
- **Notifiche:** gli avvisi di esito utilizzano `role="status"` con prefisso testuale esplicito (`<strong>Errore:</strong>` o `<strong>Fatto:</strong>`) affinché il significato non dipenda dal solo colore;
- **Tabelle:** tutte dotate di `<caption>`, celle di intestazione con `scope="col"` e `scope="row"`;
- **Form:** ogni campo ha una `<label for="...">` esplicita, i gruppi logici sono racchiusi in `<fieldset>` con `<legend>`, gli errori sono collegati ai campi tramite `aria-describedby`;
- **Grafico SVG:** il grafico degli incassi è generato lato server ed è accompagnato da una tabella riassuntiva equivalente, garantendo l’accessibilità del dato anche senza la rappresentazione visiva;
- **Contrasto:** rapporto minimo verificato di 5.02:1 su tutto il sito, superiore al requisito WCAG AA di 4.5:1;
- **Tema:** il cambio tema è accessibile con `aria-label` sui pulsanti e funziona senza JavaScript.

4.4.1 Aiuti per lo screen reader

- quando presenti termini in lingua inglese (nomi dei panini come “Cheeseburger”, “Bacon Burger”, “Chicken Wings”, ecc.), viene dichiarato l’attributo `lang="en"` per garantire la corretta pronuncia da parte dello screen reader;
- le abbreviazioni (CAP, WCAG) sono marcate con `<abbr title="...">`;
- gli elementi decorativi (icone SVG) hanno `aria-hidden="true"`;
- i tag HTML5 semantici (`<header>`, `<main>`, `<nav>`, `<footer>`, `<article>`, `<section>`) sono utilizzati in modo coerente per facilitare la navigazione per landmark.

4.4.2 Compatibilità responsive

Il sito adotta un layout fluido basato su unità relative (`em`, `rem`, `%`) e non presenta mai scorrimento orizzontale a qualsiasi larghezza di viewport. Il breakpoint principale è a 48 em (768 px), gestito dal foglio `mobile.css` collegato con `media="screen and (max-width: 48em)"`.

5 Test effettuati

5.1 Accessibilità e validazione

Per verificare l’accessibilità, la correttezza sintattica e le prestazioni del sito sono stati utilizzati sia strumenti automatici sia test manuali, in combinazione per garantire una copertura completa.

5.1.1 Strumenti automatici

- **W3C Nu HTML Validator** (`vnu.jar`): validazione HTML5 di tutte le pagine pubbliche e autenticate, integrata nel workflow GitHub Actions (`qualita.yml`). Esito: **0 errori** su tutte le pagine;
- **W3C CSS Validator** (API Jigsaw): validazione dei 3 fogli di stile tramite API remota nel workflow CI. Esito: **0 errori**;
- **xmllint**: verifica della conformità XML del markup HTML5, integrata nel workflow CI;
- **Pa11y v8**: test automatico di accessibilità WCAG 2.1 AA su tutte le pagine, incluse quelle autenticate (con login automatizzato per i tre ruoli), integrato nel workflow CI;
- **Google Lighthouse v12**: audit automatico di prestazioni, accessibilità, best practice e SEO su tutte le pagine pubbliche, con soglia minima di 90 punti per categoria;
- **WAVE by WebAIM**: estensione browser per l'analisi visuale dell'accessibilità, utilizzata durante lo sviluppo;
- **Silktide Accessibility**: estensione browser per la verifica dei contrasti e della struttura semantica.

5.1.2 Test manuali

- controllo della corretta struttura e gerarchia degli heading;
- verifica dell'uso appropriato dei tag semantici;
- controllo della coerenza e completezza degli attributi `alt` sulle immagini;
- navigazione completa da tastiera su tutte le pagine del sito;
- verifica del corretto funzionamento dei form e della gestione degli errori;
- verifica del funzionamento completo del sito con JavaScript disabilitato;
- controllo del layout responsive su diverse risoluzioni;
- verifica della leggibilità e del contrasto dei colori in entrambi i temi;
- test di stampa delle pagine per verificarne la formattazione su carta;
- test di accessibilità con screen reader NVDA;
- compatibilità con i principali browser: Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari, Opera;
- compatibilità con diversi sistemi operativi: Windows 10/11, Android, iOS.

5.2 Integrazione continua (CI/CD)

Il repository utilizza due workflow GitHub Actions:

1. `qualita.yml`: eseguito ad ogni push e pull request, comprende tre job paralleli:
 - **markup**: validazione HTML5 (`vnu.jar`), conformità XML (`xmllint`), validazione CSS (W3C API);
 - **accessibilita**: Pally WCAG 2.1 AA su tutte le pagine con autenticazione automatizzata;

- **prestazioni:** Google Lighthouse con soglia ≥ 90 su performance, accessibilità, best practice e SEO;
2. **deploy-tecweb.yml:** deploy automatico sul server universitario tramite tunnel SSH attraverso il gateway `paolotti.studenti.math.unipd.it`, con rsync, correzione permessi e importazione opzionale del database.

5.3 Analisi dei falsi positivi

L'analisi degli strumenti automatici non ha evidenziato falsi positivi significativi. Le uniche segnalazioni riscontrate riguardano avvisi (non errori) del validatore CSS sulle variabili custom (`custom properties`), che sono conformi allo standard CSS e correttamente ignorate dal workflow CI.

5.4 Screen reader

L'accessibilità del sito è stata testata con lo screen reader NVDA (software gratuito e open-source), verificando la corretta lettura e interpretazione di tutti i contenuti e degli elementi interattivi. I test hanno incluso il controllo dell'ordine di navigazione da tastiera, della corretta associazione tra etichette e campi di input, dell'interpretazione dei ruoli e degli attributi ARIA, della lettura strutturata di tabelle e form, e della corretta pronuncia dei termini in lingua inglese. Le verifiche non hanno evidenziato criticità.

6 Ambiente di sviluppo

L'ambiente di sviluppo locale replica l'ambiente di produzione del server universitario (Ubuntu 22.04, PHP 8.1, MariaDB 10.6) tramite Docker Compose con tre servizi:

- **web:** PHP 8.1 Apache con moduli `rewrite`, `headers`, `deflate`, `expires`;
- **db:** MariaDB 10.6 con volume persistente e auto-import dello schema;
- **phpmyadmin:** interfaccia grafica per la gestione del database.

7 Organizzazione del gruppo

Il lavoro è stato organizzato suddividendo le attività in modo equo tra i membri del gruppo, con ciascun componente coinvolto in tutte le fasi del progetto (HTML, CSS, PHP, JavaScript, database, testing). Il coordinamento è avvenuto tramite GitHub con issue, pull request e code review reciproche.

7.1 Divisione dei compiti

- **[DA COMPILARE: Nome Cognome]:**
 - HTML/CSS: pagine Home, Menu, Dettaglio prodotto, Servizi
 - PHP/JavaScript: funzionalità inerenti alle pagine sviluppate
 - DB: progettazione schema e popolamento dati iniziali
 - Testing e validazione: WAVE, Silktide, W3C Validator e NVDA per le relative pagine
 - Relazione tecnica
- **[DA COMPILARE: Nome Cognome]:**

- HTML/CSS: pagine Chi siamo, Sedi, Contatti, Accessibilità, Privacy
- PHP/JavaScript: funzionalità inerenti alle pagine sviluppate
- DB: ottimizzazione query e vincoli di integrità
- Testing e validazione: WAVE, Silktide, W3C Validator e NVDA per le relative pagine
- Relazione tecnica
- **[DA COMPILARE: Nome Cognome]:**
 - HTML/CSS: pagine Carrello, Pagamento, Ricevuta, Ordini, Prenotazioni
 - PHP/JavaScript: funzionalità inerenti alle pagine sviluppate
 - DB: gestione transazioni e concorrenza magazzino
 - Testing e validazione: WAVE, Silktide, W3C Validator e NVDA per le relative pagine
 - Relazione tecnica
- **[DA COMPILARE: Nome Cognome]:**
 - HTML/CSS: pagine Pannello di controllo (prodotti, categorie, sedi, utenti, contatti)
 - PHP/JavaScript: funzionalità inerenti alle pagine sviluppate
 - DB: backup e deploy automatizzato
 - CI/CD: configurazione workflow GitHub Actions (qualita.yml, deploy-tecweb.yml)
 - Testing e validazione: WAVE, Silktide, W3C Validator e NVDA per le relative pagine
 - Relazione tecnica

8 Note

Il sito è stato sviluppato senza l'utilizzo di framework, librerie JavaScript o CSS esterni. L'unica dipendenza esterna è il font Archivo, distribuito sotto licenza libera Open Font License e caricato localmente dai file della cartella `src/styles/caratteri/`.

Il cambio tema chiaro/scuro rispetta automaticamente la preferenza di sistema dell'utente tramite la media query `prefers-color-scheme`, ma consente di sovrascriverla manualmente tramite i pulsanti nel layout. La preferenza manuale viene salvata in un cookie e gestita interamente lato server, senza dipendenze JavaScript.

Per lo sviluppo è stato adottato un approccio di tipo **Progressive Enhancement**: tutte le funzionalità critiche del sito (ordinazione, prenotazione, gestione) funzionano completamente anche con JavaScript disabilitato. Lo scripting aggiunge miglioramenti incrementali come l'invio AJAX dei form, la gestione dinamica del pulsante "Torna su" e la visualizzazione condizionale dei campi nel form di pagamento.